

Assignment

Due: Monday, 12 October 2026, 11:59 PM (GMT+8)

Weight: 30% of the unit

Your task for this assignment is to design, code (in C89 version of C) and test a program. In summary, your program will:

- Able to create dynamically-allocated 2D char array to make a simple ASCII-based game.
- Receive user input to control the game.
- Use pre-written colour generator to add background/foreground colour in the game.
- Write a proper makefile. Without the makefile, we will assume it cannot be compiled and it will negatively affect your mark.
- Extract information from a text file to initialize the game.
- Utilise linkedlist data structure to keep track the game progress, allowing the Player to undo the steps taken.

1 Code Design

You must comment your code sufficiently in a way that we understand the overall design of your program. Remember that you cannot use `/**` to comment since it is C99-specific syntax. You can only use `/*.....*/` syntax.

Your code should be structured in a way that each file has a clear scope and goal. For example, `main.c` should only contain a main function, `game.c` should contain functions that handle and control the game features, and so on. Basically, functions should be located on reasonably appropriate files and you will link them when you write the makefile. DO NOT put everything together into one single source code file. Make sure you use the header files and header guard correctly. NEVER `#include .c` files directly, instead `#include .h` files only.

Make sure you free all the memories allocated by the `malloc()` function. Use `valgrind` to detect any memory leak and fix them. Memory leaks might not break your program, but penalty will still be applied if there is any. If you do not use `malloc` at all, you will lose marks.

Please be aware of our coding standard (can be found on Blackboard and the marking criterias) Violation of the coding standard will result in penalty on the assignment. Keep in mind that it is possible to lose significant marks with a fully-working program that is written messily, has no clear structure, full of memory leaks, and violates many of the coding standards. The purpose of this unit is to teach you a good habit of programming.

2 Academic Integrity

This is an assessable task, and as such there are strict rules. You must not ask for or accept help from anyone else on completing the tasks. You must not show your work at any time to another student enrolled in this unit who might gain unfair advantage from it. These things are considered plagiarism or collusion. To be on the safe side, never release your code to the public.

Do NOT just copy some C source codes from internet resources (including AI tools such as ChatGPT). If you get caught, it will be considered plagiarism. If you put the reference, then that part of the code will not be marked since it is not your work.

Staff can provide assistance in helping you understand the unit material in general, but nobody is allowed to help you solve the specific problems posed in this assignment. The purpose of the assignment is for you to solve them on your own, so that you learn from it. Please see Curtin's Academic Integrity website for information on academic misconduct (which includes plagiarism and collusion).

The unit coordinator may require you to attend quick interview to answer questions about any piece of written work submitted in this unit. Your response(s) may be referred to as evidence in an Academic Misconduct inquiry. In addition, your assignment submission may be analysed by systems to detect plagiarism and/or collusion.

3 Task Details

3.1 Quick Preview

First of all, please watch the supplementary videos on the Assignment link on Blackboard. These videos demonstrate what we expect your program should do. You will implement a simple game that you can play on Linux terminal. Further details on the specification will be explained on the oncoming sections.

3.2 Command Line Arguments

Please ensure that the executable is called “**dungeon**”. Your executable should accept only one more command-line parameter/argument:

`./dungeon <map_file>`

<map_file> is the text file name containing the information to initiate your game. It is your responsibility to check whether the file can be opened properly. You can **assume the content of the file is valid** to initiate the game.

If the amount of the arguments are too many or too few, your program should print a message on the terminal and end the program accordingly (do NOT use `exit()` function).

Note: Remember, the name of the executable is stored in variable `argv[0]`.

3.3 Main Interface

Once you run the program, it should clear the terminal screen (watch the video about the method to clear the screen) and print the 2D map containing all the objects: Block, Ladder, Danger Zone, Player, and the Goal with correct colours and surrounded by map borders.

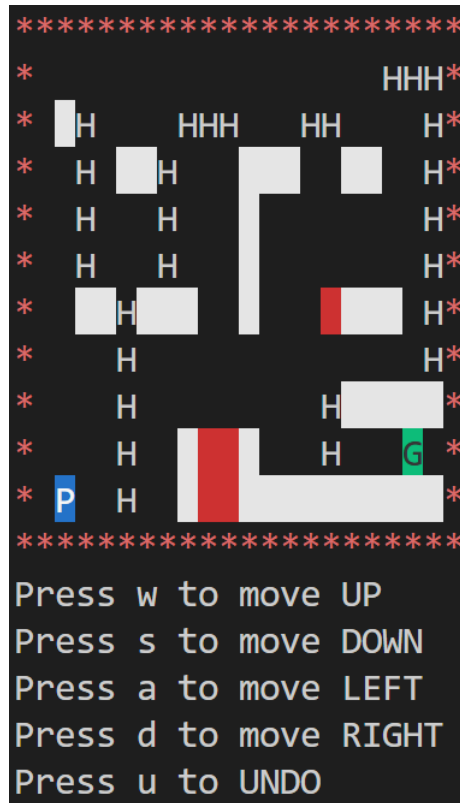


Figure 1: Main visual interface of the game.

You can type the command to move the Player. The main objective is to move the Player to the Goal while avoiding the Danger Zone and preventing them from falling to their death. Every time the user inputs the command, the program should update the map accordingly and refresh the screen (clear the screen and reprint the map). Please refer to Section 3.6.1 for more detail on the user input.

3.4 Characters on the Map

You have to use the correct characters on the game interface:

- Char '*' for the **map border**.
- An empty space ' ' char for the **Block** and **Danger Zone**.
- Char 'H' for the **Ladder**.
- Char 'P' for the **Player**.
- Char 'G' for the **Goal**.

Furthermore, one object has **foreground colour**:

- **Map border** has **red** foreground colour.

Some objects have **background colour**:

- **Block** has **white** background colour.
- **Danger Zone** has **red** background colour.
- **Player** has **blue** background colour.
- **Goal** has **green** background colour.

3.5 File I/O

Please watch the supplementary video about the File I/O. The text file will look like this:

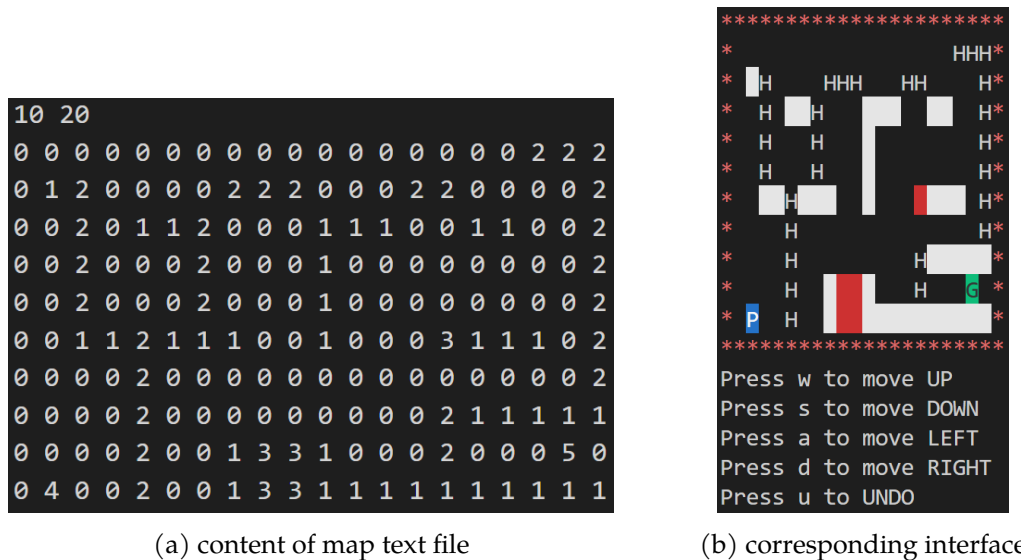


Figure 2: This is an example of the map text file and the game interface created from it.

The two integers on the first line are the playable **row** and **column** map size respectively. These map sizes do not include the border surrounding the map. Afterwards, the remaining lines are the content of the 2D map array. You can assume the text file contains valid information with valid datatype. Each integer represents:

- 0 represents ' ' (An empty space)
- 1 represents ' ' (Block, also an empty space, but with white background)
- 2 represents 'H' (Ladder)
- 3 represents ' ' (Danger Zone, also an empty space, but with red background)
- 4 represents 'P' (Player)
- 5 represents 'G' (Goal)

Note: Keep in mind that we will use our own map text file when we mark your assignment. So, please make sure your file I/O implementation works with various map files. Feel free to create your own map file.

3.6 Gameplay

Please watch the supplementary videos for visual demonstration. The main objective is to move the Player to reach the Goal while avoiding the Danger Zone and preventing the Player from falling too high to their death.

3.6.1 User Input

The user only needs to type 1 character for each command (lower case). Here is the list of the possible commands:

- 'w' moves the Player one grid upward.
- 's' moves the Player one grid downward.
- 'a' moves the Player one grid to the left.
- 'd' moves the Player one grid to the right.
- 'u' undoes the movement of the Player. If the Player falls from a survivable height, then the Undo button will restore the Player's position to just before the fall.
- Any other character should be ignored, and nothing changes on the map. Feel free to print a warning message such as "invalid key".

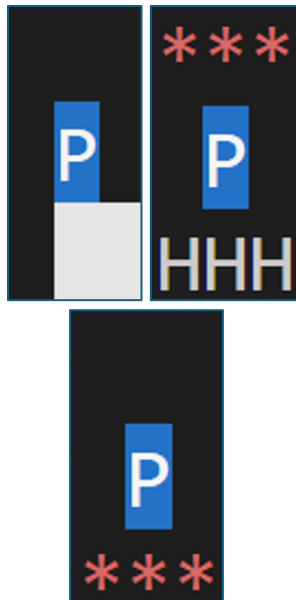
Your program should be able to receive each input without pressing "enter" key everytime. Please refer to the supplementary video for a sample code to disable the Echo and Canonical feature on Linux terminal temporarily. If your terminal is stuck on this mode, you only need to re-open a new terminal.

Note: Every action should update the 2D map array, clear the terminal screen and reprint the map.

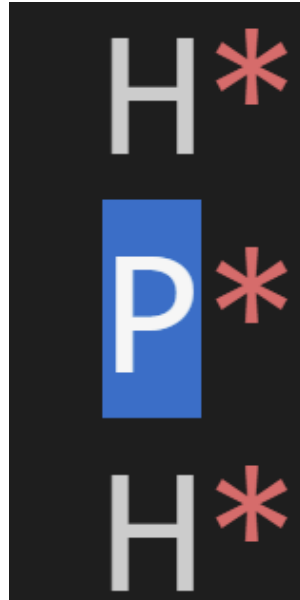
3.6.2 Player Movement

PLAYER STATES In this game, the Player can be in one of three states:

- Standing on a solid object. (Block, Ladder, or bottom of the map)
- Holding onto a Ladder. (The Player and Ladder at the same position)
- Not standing on any object. (Falling down)



(a) Standing on a solid object



(b) Holding onto a ladder



(c) Not standing on any object

Figure 3: The three possible states of the Player.

HORIZONTAL MOVEMENT The Player can move horizontally while standing on solid a object OR holding onto a ladder. The Player cannot move through Blocks or the border of the map (left and right border). If the Player steps off a solid object into empty space, they will begin falling.

VERTICAL MOVEMENT The Player can move vertically while holding onto a Ladder. If Player is standing on top of a Ladder, they can move down the Ladder. The Player cannot move through Blocks or the border of the map (top and bottom border). The Player can step off a ladder at any point, causing them to fall.

FALLING ANIMATION At any point, if the Player is not standing on a solid object, the downward falling animation should be played. Once The Player begins falling, the animation cannot be interrupted until they land on a solid object OR reach the Danger Zone. A reasonable frame transition time for the falling animation is approximately 0.1-0.2 seconds per frame.

Note: Please watch the General Supplementary Video about creating animation and Supplementary Video about Player falling animation.

3.6.3 Winning/Losing the Game

The game will continue playing until either the Player manages to **reach the Goal (winning)** OR the Player **touches the Danger Zone OR falls too far (losing)**. A fall is considered too high if the Player falls **at least 5 grids** before landing. You can print message such as "You win!" or "You lose!" when the game ended. Please do **NOT** use **exit()** or **break** to end the program. You must free all the malloc'ed memories and ensure the program reaches the end of the main function properly.

Note: If the Player falls too far but lands on the Goal, the game is considered won.

3.7 Struct and Generic Linked List to implement UNDO feature

In this section, you will need to write a **generic linked list** with appropriate struct to implement **UNDO** feature. This feature is triggered when the user enters the key 'u'. Everytime the key is pressed, the Player will move back to their previous position. The Player can use the UNDO feature as long as the game is not yet over (Not losing AND not winning yet). The Player cannot use the UNDO feature during the falling animation. It should be possible to keep UNDO-ing up to the initial state of the game.

Please refer to supplementary video for some advices regarding this implementation. The summary is that you can use the linkedlist node to store any information that you deem necessary. For example, you can store the previous Player 'P' location (This is only one example, you can store more information if you need it). You will need to create a new linkedlist node every time the Player attempts to move (with malloc()) and store it in the linkedlist. When the Player attempts to do invalid move (e.g moving to a Block OR outside map border), it does not count as a movement, and the linkedlist should not record that move.

Note: The falling animation should not be recorded in the linkedlist. Instead, record the last position of the Player just before the fall.

When the Player presses 'u', your program should be able to retrieve the most recent node and update the map to the previous state. If game state is restored to its initial state (i.e. the state when the game was first started), pressing 'u' should **NOT** do anything. One possible way to confirm this is by checking if the linkedlist is empty. (This depends on your implementation. Implement what you need to achieve this task.)

Note: To get full mark on this section, you need to implement a **fully generic linkedlist**. If you implement a non-generic linkedlist, you still can get partial mark. Please do not just copy the whole linkedlist code from someone else because it might lead to collusion academic misconduct.

3.8 Makefile

You should manually write the makefile according to the format explained in the lecture and practical. It should include the Make variables, appropriate flags, appropriate targets, correct prerequisites, and clean rule. We will compile your program through the makefile. If the makefile is missing, we will assume the program cannot be compiled, and you will lose marks. **DO NOT** use the makefile that is auto-generated by the VScode. Write it yourself.

3.9 Assumptions

For simplification purpose, these are assumptions you can use for this assignment:

- The game always starts with the Player standing on a solid object such as a Block, Ladder, or the bottom of the map.
- The Goal should be reachable. (There is a clear path from the Player to the Goal)
- The Goal must be placed on top of a solid object. (NOT floating in mid-air)
- The contents inside the map file are valid. (If there is NO error when opening it)
- The size of the map in the text file is at least 5 rows AND at least 5 columns.
- All game objects exist. There will be ONLY one Player and ONLY one Goal. There will be at least one Block, one Ladder, and one Danger Zone.
- The size of the map will be reasonable to be displayed on terminal. For example, we will not test the map with gigantic size such as 300 x 500.
- You only need to handle lowercase inputs ('w', 's', 'a', 'd', 'u'). The other keys should be ignored (feel free to add warning message).

Note: When you create your own custom maps for testing, please make sure to follow these assumptions.

4 Marking Criteria

This is the marking distribution for your guidance:

- Properly structured makefile according to the lecture and practical content (**5 marks**)
- Program can be compiled with the makefile and executed successfully without immediate crashing and showing reasonable output (**5 marks**)
- Usage of header guards and reasonable in-code commenting (**2 marks**)
- The whole program is readable and has reasonable framework. Multiple files are utilised with reasonable category. If you only have one c file for the whole assignment (not counting terminal.c, newsleep.c, and color.c), you will get zero mark on this category. (**3 marks**)
- Avoiding bad coding standard. (**5 marks**) Please refer to the coding standard on Blackboard. Some of the most common mistakes are:
 - Using global variables
 - Calling exit() function to end the program abruptly
 - Using “break” NOT on the switch case statement
 - Using “continue”
 - Using goto
 - Having multiple returns on a single function
 - #include the .c files directly instead of the .h files
- No memory leaks (**10 marks**) Please use valgrind command to check for any memory leak. If your program is very sparse OR does not use any malloc(), you will get zero mark on this category.

• FUNCTIONALITIES:

- Correct command line arguments verification with proper response. This includes detecting the success/failure of opening the text file. **(5 marks)**
- File IO is done successfully to retrieve all game information from the map text file. Proper memory allocation for the 2D map array. **(10 marks)**
- Able to clear the screen and re-print the map (with correct symbols and correct foreground and background colours for all objects) on every action. **(5 marks)**
- Able to move the Player with the keyboard input from the user without pressing 'Enter' key. (and cannot move through Block and map border) **(5 marks)**
- Player can move horizontally while standing on solid objects OR by holding onto the Ladder. **(5 marks)**
- Player can move vertically using the Ladder. **(5 marks)**
- Falling animation is implemented correctly when the Player is not standing on any solid object. **(10 marks)**
- The Player wins when they reach the Goal. **(5 marks)**
- The Player loses when they touch any Danger Zone (even while falling) **OR** they fall for at least 5 grids in height and lands on a non-Goal object. **(5 marks)**
- Able to UNDO the game utilizing linkedlist. **(15 marks)** (If linkedlist is NOT generic, only 10 marks max)

Note: Remember, if your program fails to demonstrate that the functionality works, then the mark for that functionality will be capped at 50%. If your program does not compile at all OR just crashed immediately upon running it, then the mark for all the functionalities will be capped at 50% (For this assignment, it means maximum of 35 out of 70 marks on functionalities). You then need describe your code clearly on the short report on the next section.

5 Short Report

If your program is incomplete/imperfect, please write a short report explaining what you have done on for each functionality. Inform us which function on which file that is related to that functionality. This report is NOT marked, but it will help us a lot to mark your assignment. Poorly-written report will not help us to understand your code. Please ensure your report reflects your work correctly (no exaggeration). Dishonest report will lead to academic misconduct.

6 Final Check and Submission

After you complete your assignment, please make sure it can be compiled and run on our Linux lab environment. If you do your assignment on other environments (e.g on Windows operating system), then it is your responsibility to ensure that it works on the lab environment. In the case of incompatibility, it will be considered “not working” and some penalties will apply. You have to submit a **single zip file** containing ALL the files in this list in **ONE FOLDER**:

- **Declaration of Originality Form** – Please fill this form digitally and submit it. This is an important document for assignment submission.
- **Your essential assignment files** – Submit all the .c & .h files and your makefile. Please do not submit the executable and object (.o) files, as we will re-compile them anyway. Do not create any sub-directory within the directory. Every file should exist within the same directory.
- **Brief Report** (if applicable) - If you want to write the report about what you have done on the assignment, please save it as a PDF or TXT file.

Note: Please compress all the necessary files into a **SINGLE zip file**. So, your submission should only has one zip file. If you do not compress the files, we reserve the right to refuse your submission. If you get an **extension**, please **do NOT submit the early draft** to the Blackboard because we might accidentally mark the earlier version of your work. Only submit to Blackboard once you have the final version of your assignment.

The name of the zipped file should be in the format of:

<your-tutor-name>_<student-ID>_<your-full-name>_Assignment.zip

Example: Antoni_12345678_Michael-Bay_Assignment.zip

If you forget your tutor’s name, please put the lecturer’s name. Please make sure your submission is complete and not corrupted. You can re-download the submission and check if you can compile and run the program again. Corrupted submission will receive instant zero. Late submission will receive zero mark. You can submit it multiple times, but only your latest submission will be marked. Therefore, please make sure the latest submission is the complete code.

End of Assignment